VLSI Design

Chapter 4:
Delay

Delay Definitions

- when an input changes, the output will retain its old value for at least the contamination delay and take on its new value in at most the propagation delay.

- *Propagation delay time*, t_{pd} = maximum time from the input crossing 50% to the output crossing 50%
- *Contamination delay time*, t_{cd} = minimum time from the input crossing 50% to the output crossing 50%
- *Rise time*, t_r = time for a waveform to rise from 20% to 80% of its steady-state value
- *Fall time*, t_f = time for a waveform to fall from 80% to 20% of its steady-state value
- *Edge rate*, $t_{rf} = (t_r + t_f)/2$

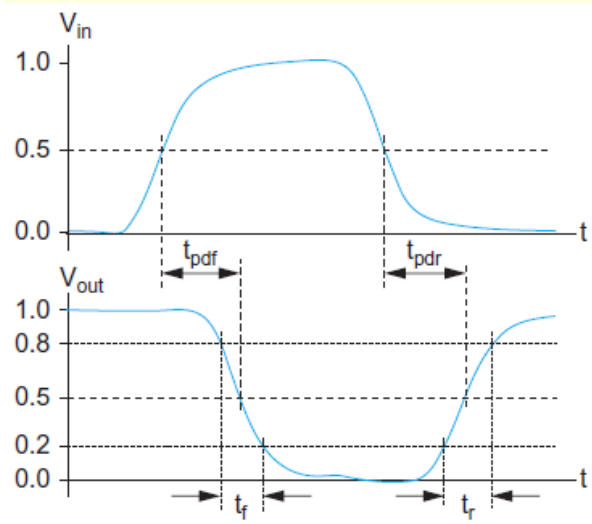
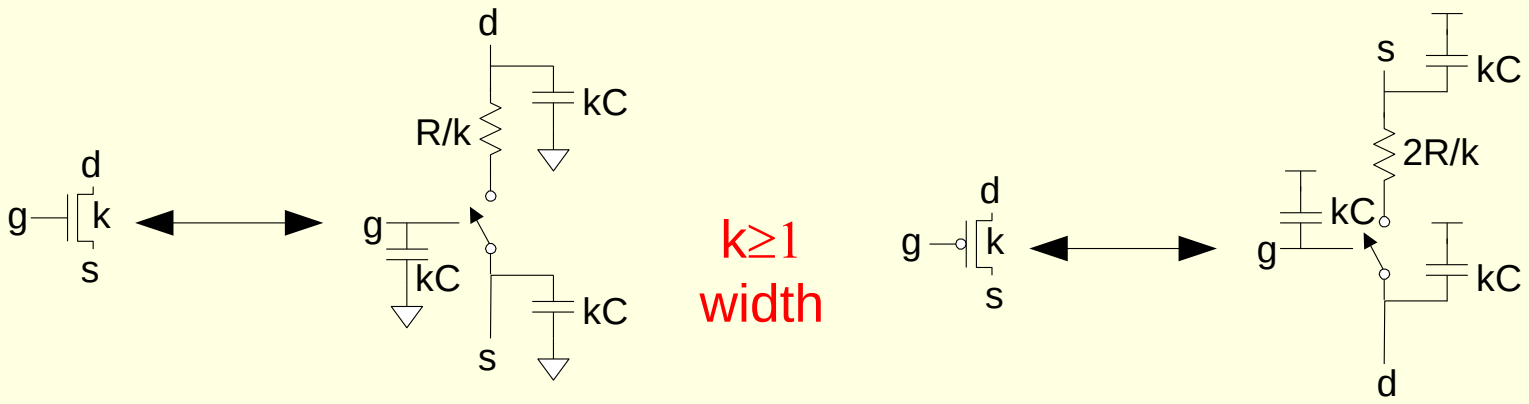


FIGURE 4.1 Propagation delay and rise/fall times

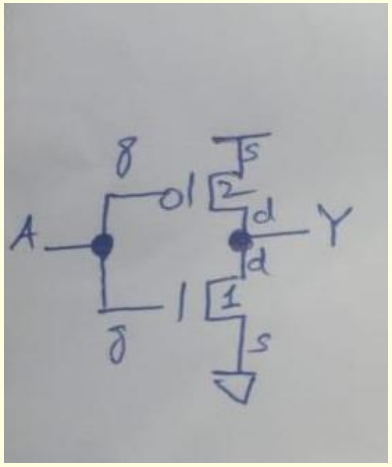
RC Delay Model

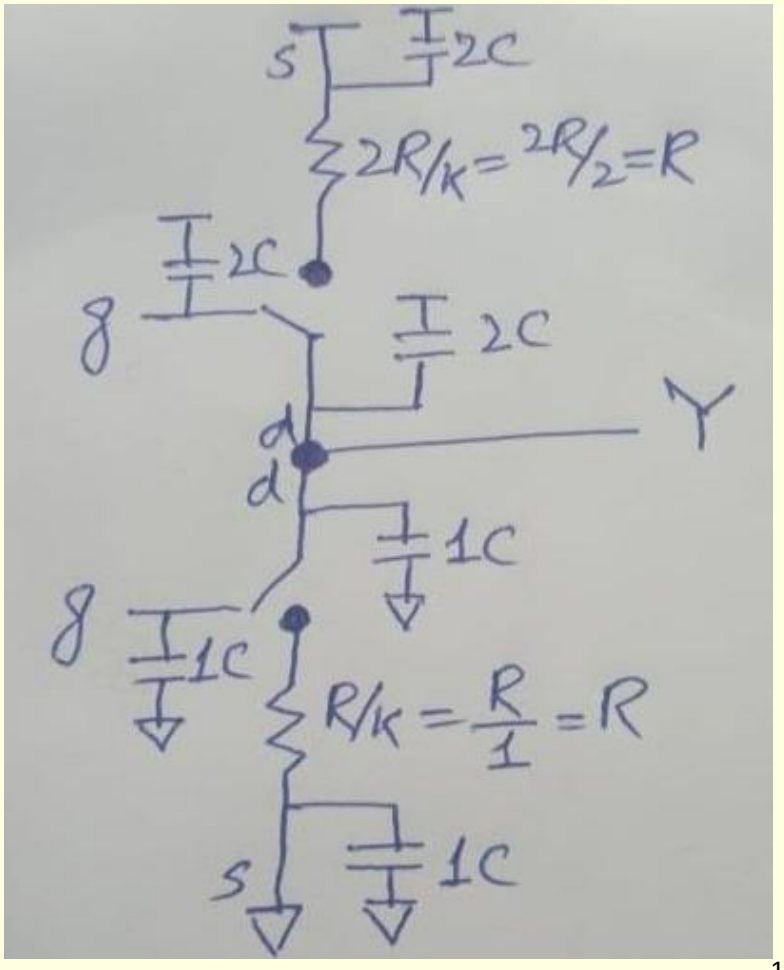
- ***RC delay models*** approximate the nonlinear transistor I-V and C-V characteristics with an average resistance and capacitance over the switching range of the gate.
- Use equivalent circuits for MOS transistors
  - Ideal switch + capacitance and ON resistance
  - Unit nMOS has resistance R, capacitance C
  - Unit pMOS has resistance 2R, capacitance C
- Capacitance proportional to width
- Resistance inversely proportional to width



RC Delay Model…Inverter

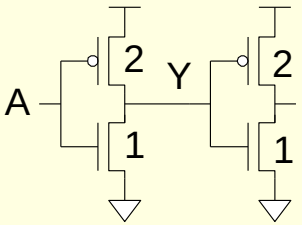
- **Drain → Source**
- Drain Diffusion Capacitance
- Ideal Switch
- Average ON Resistance
- Source Diffusion Capacitance





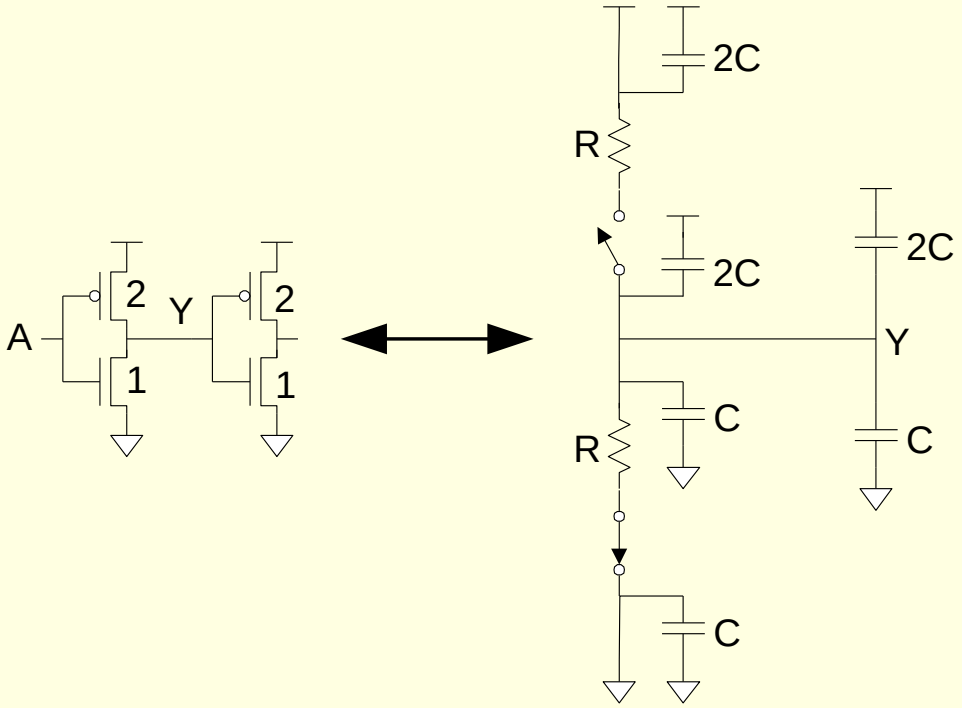
Inverter Delay Estimate

- Estimate the delay of a fanout-of-1 inverter



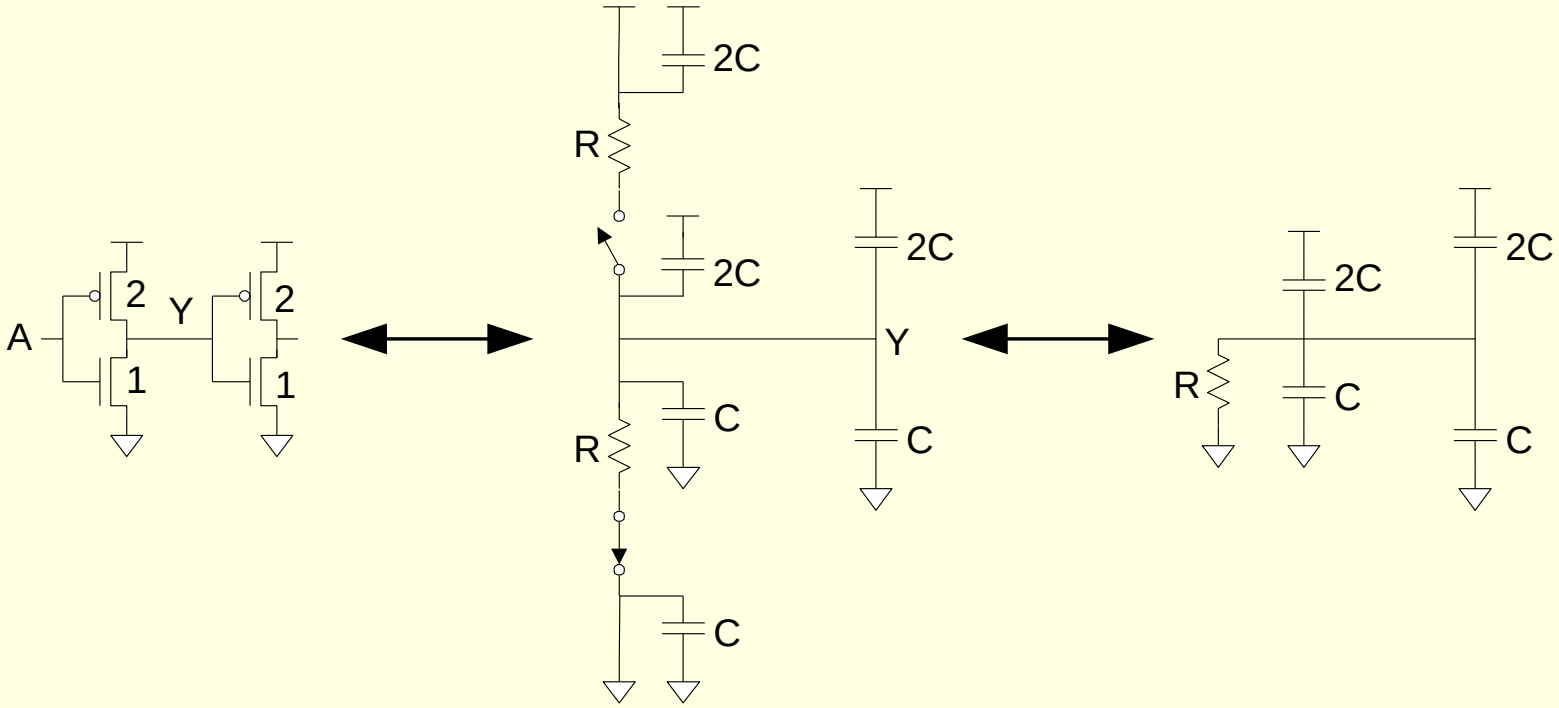
Inverter Delay Estimate

- Estimate the delay of a fanout-of-1 inverter



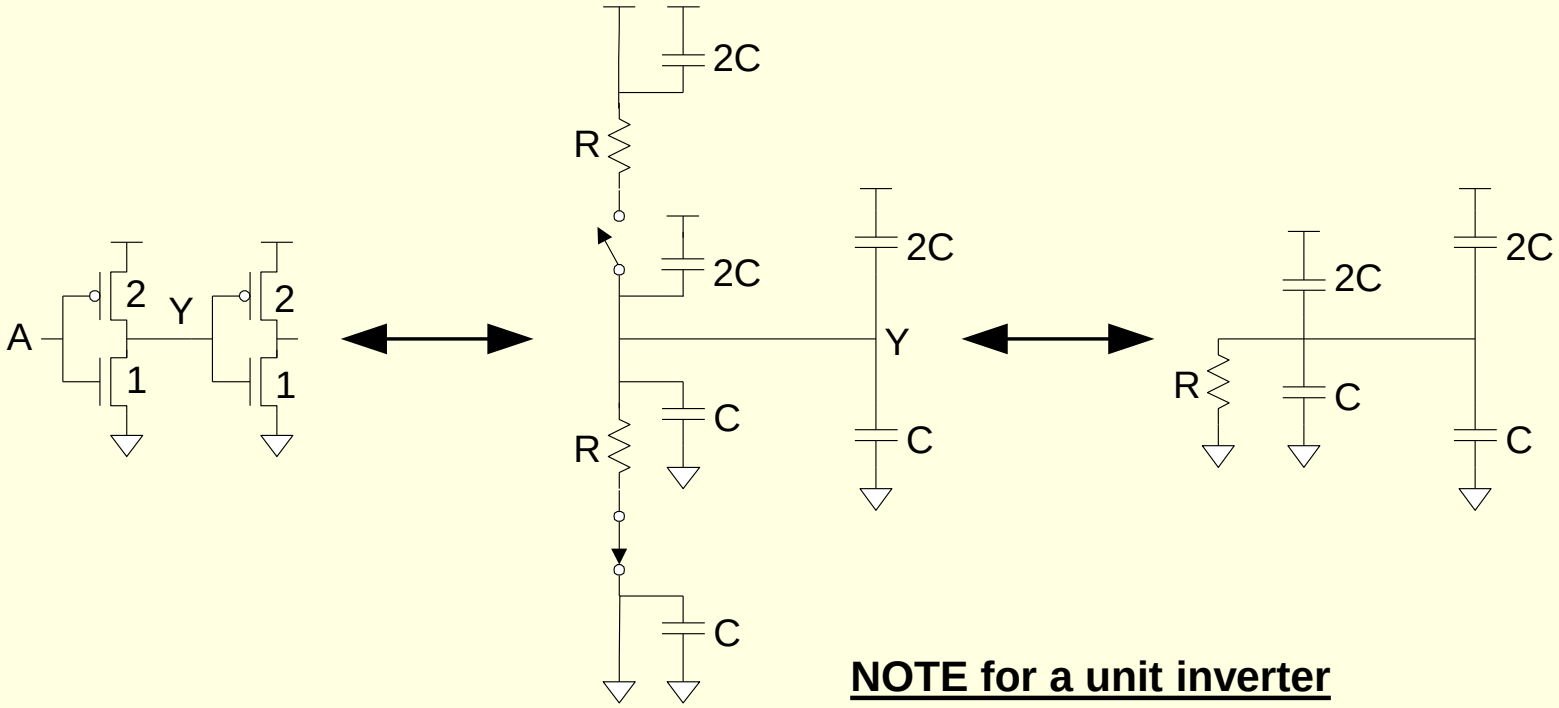
Inverter Delay Estimate

- Estimate the delay of a fanout-of-1 inverter



Inverter Delay Estimate

- Estimate the delay of a fanout-of-1 inverter



d = 6RC

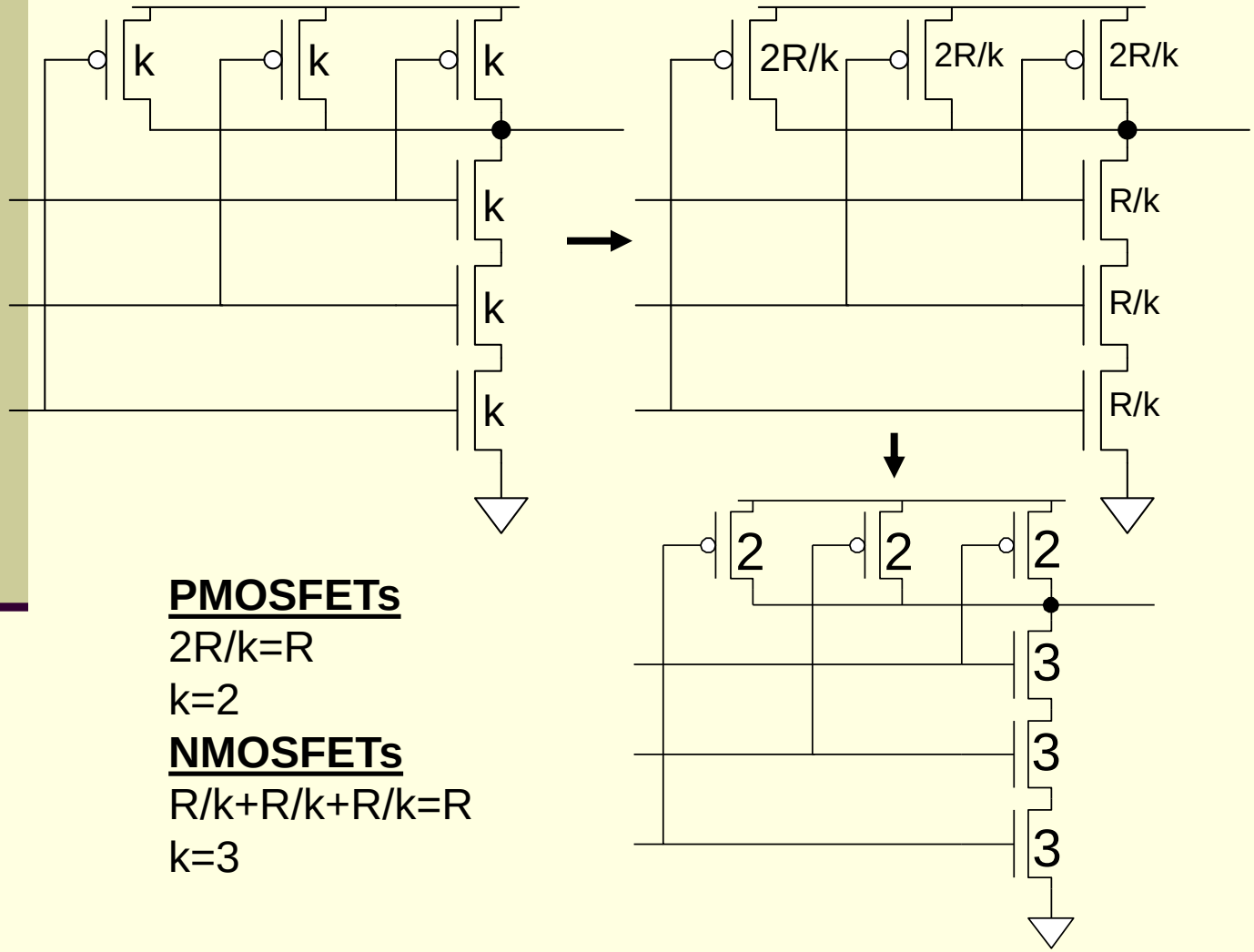
<u>NOTE for a unit inverter</u>

<u>(transistor widths PMOS 2 & NMOS 1):</u>

Effective fall resistance → *R*

Effective rise resistance → *R*

Sizing of Gates



PMOSFETs

2R/k=R

k=2

NMOSFETs

R/k+R/k+R/k=R

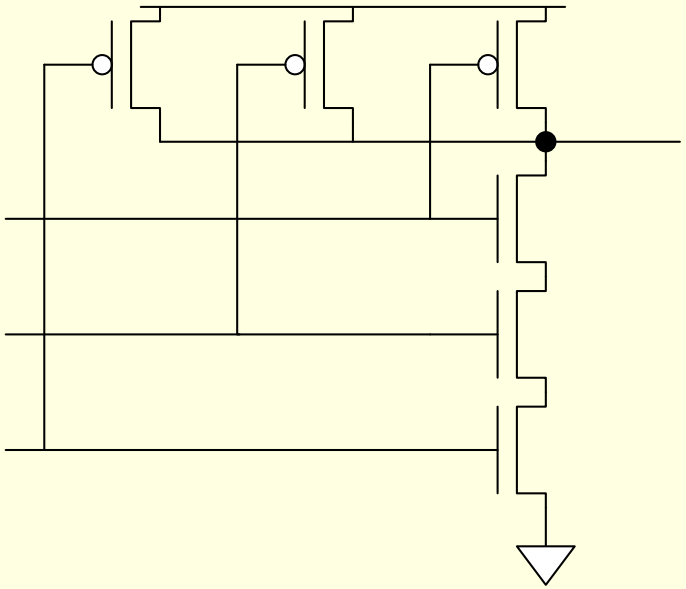
k=3

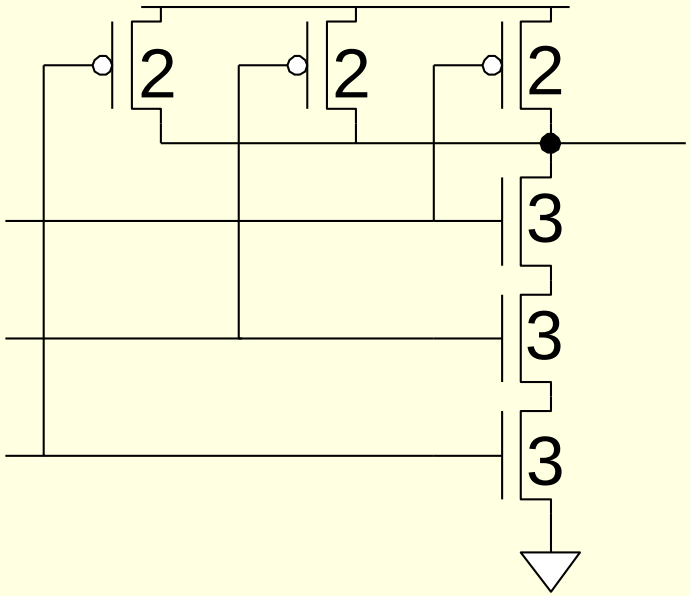
Example

- Sketch a 3-input NAND gate with transistor widths chosen to achieve effective rise and fall resistance equal to that of a unit inverter (*R).*
- Annotate the gate with its gate and diffusion capacitances. Assume all diffusion nodes are contacted.
- Then sketch equivalent circuits for the falling output transition and for the worst-case rising output transition.

Example: 3-input NAND

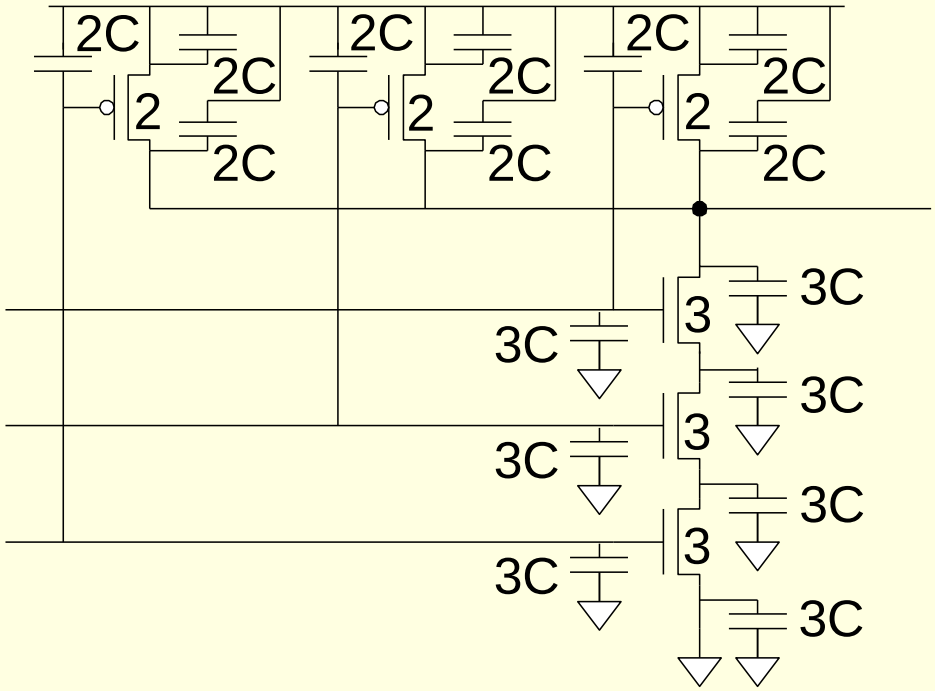
- A 3-input NAND with transistor **widths** chosen to achieve effective rise and fall resistances equal to a unit inverter (R).





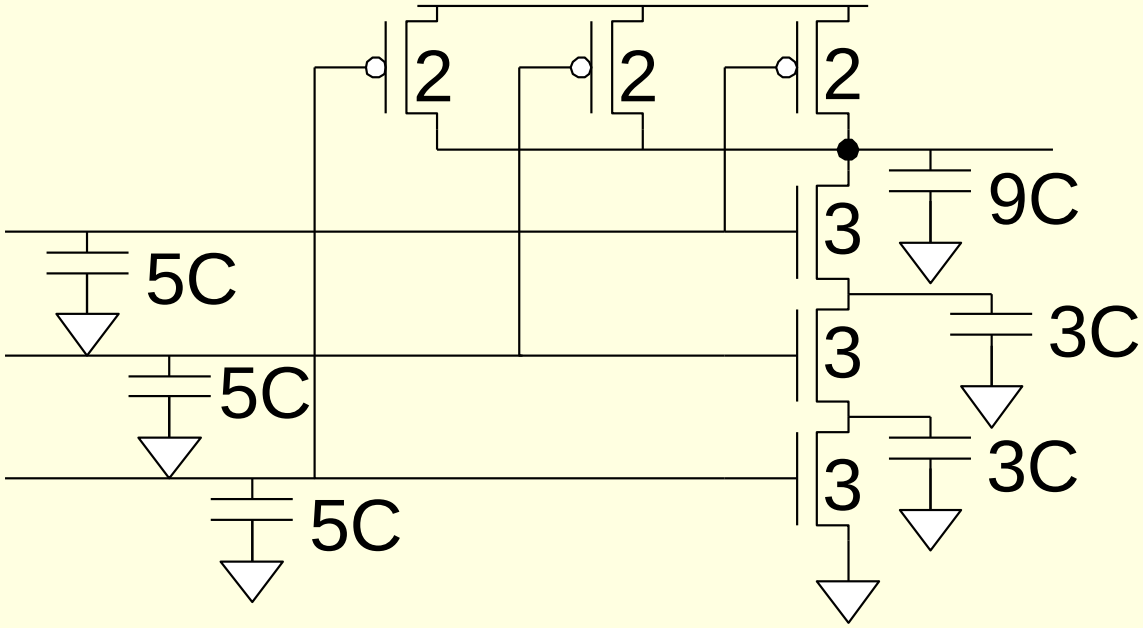
3-input NAND Caps

- Annotate the 3-input NAND gate with gate and diffusion capacitance.



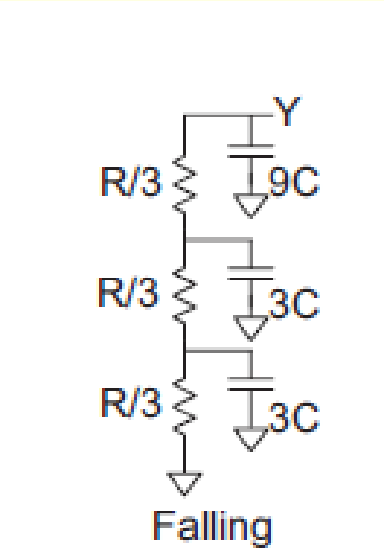
3-input NAND Caps… Continue…

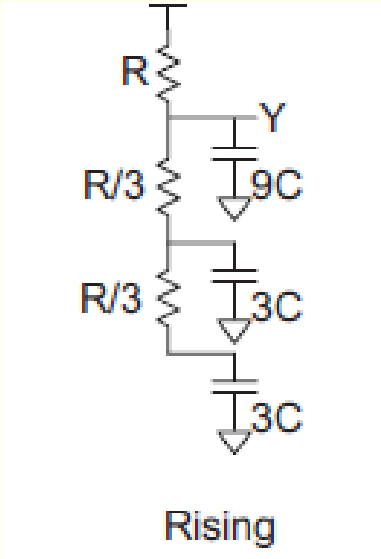
- Annotate the 3-input NAND gate with gate and diffusion capacitance.



3-input NAND Falling and Rising

- Then sketch equivalent circuits for the falling output transition and for the worst-case rising output transition.
- Falling output transition is at (111)
- Worst-case rising output transition is at (110)



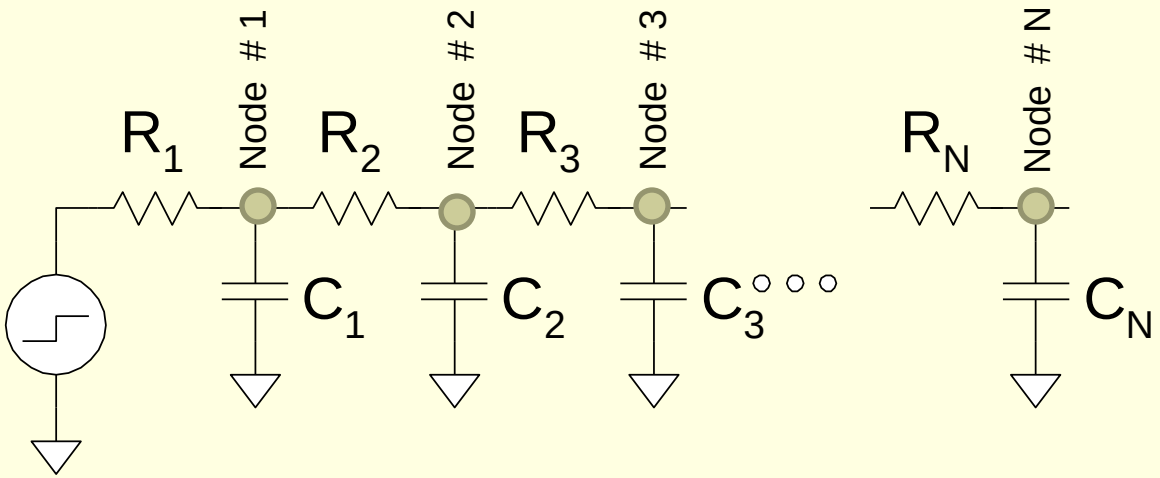


Elmore Delay Model

- Estimates Propagation Delay as the sum over each node in the ladder of the resistance between that node and the source, multiplied by the capacitance on the node:

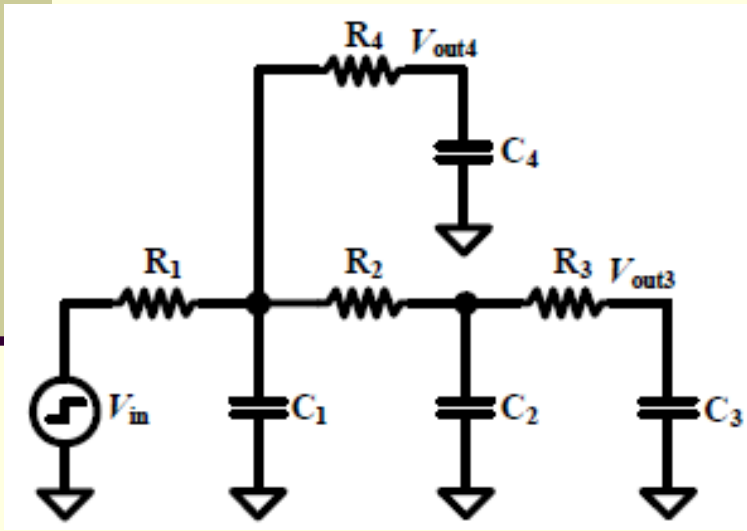
$$t_{pd} \approx \sum_{\text{nodes } i} R_{i-to-source} C_i$$

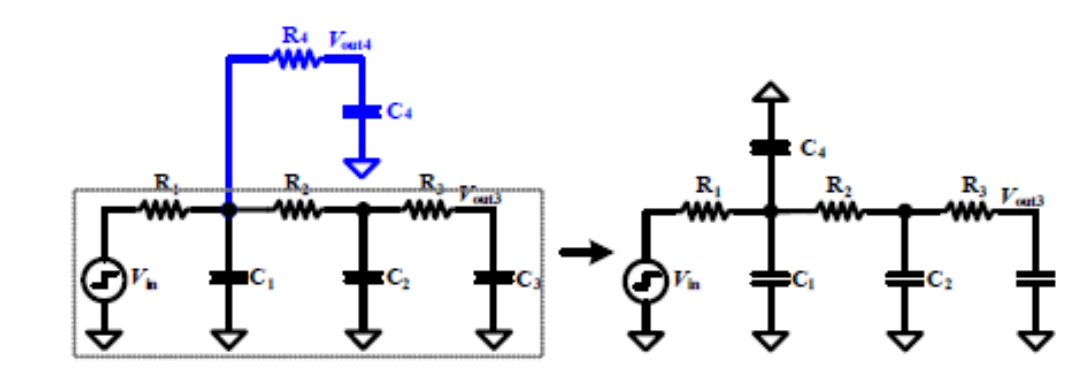
$$= R_1 C_1 + (R_1 + R_2) C_2 + \ldots + (R_1 + R_2 + \ldots + R_N) C_N$$



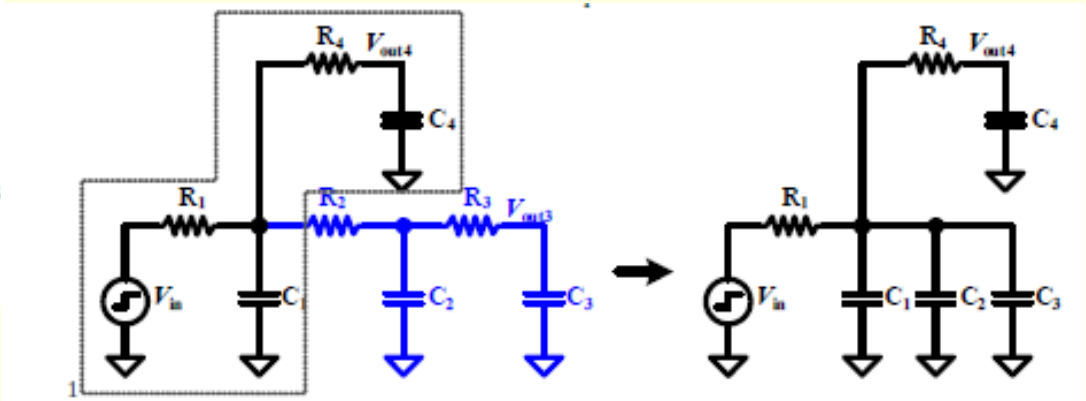
Example…Elmore Delay Model

- Find the Elmore Delay of the nodes V_{out3} and V_{out4} in the RC tree:





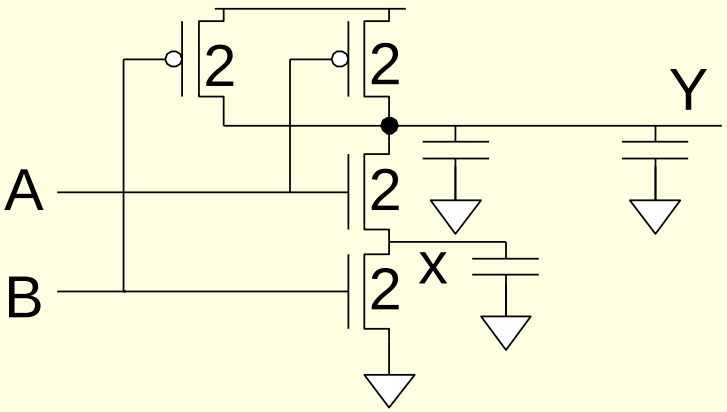
$$T_{D3} = R_1C_1 + (R_1 + R_2)C_2 + (R_1 + R_2 + R_3)C_3 + R_1C_4$$

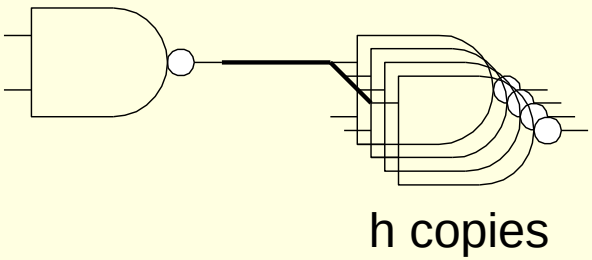


$$T_{D4} = R_1C_1 + R_1C_2 + R_1C_3 + (R_1 + R_4)C_4$$

Propagation Delay

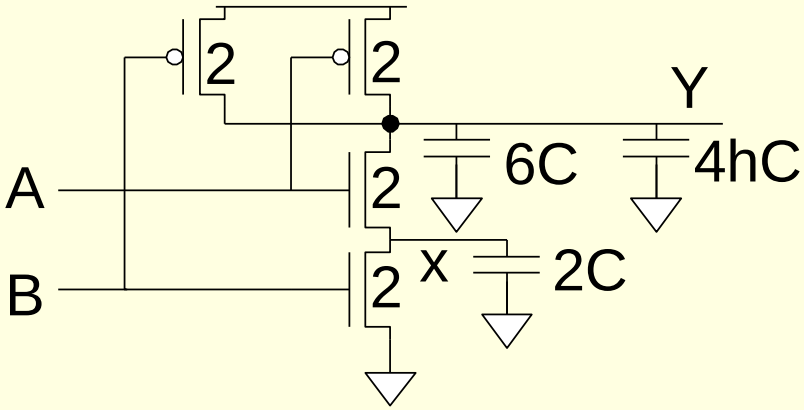
- Example: 2-input NAND
- Estimate worst-case rising and falling delay of 2-input NAND driving *h* identical gates.

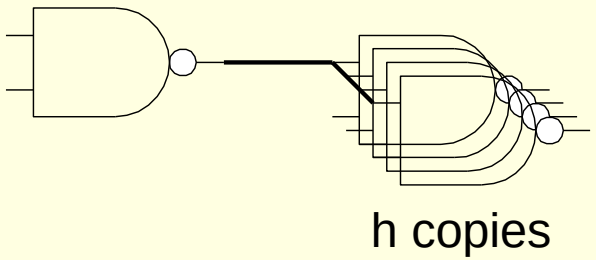




Propagation Delay

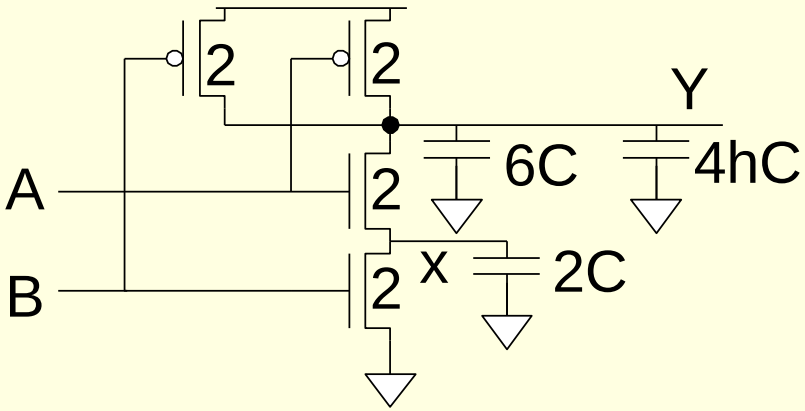
- Example: Estimate worst-case rising and falling propagation delays of a 2-input NAND driving *h* identical gates.

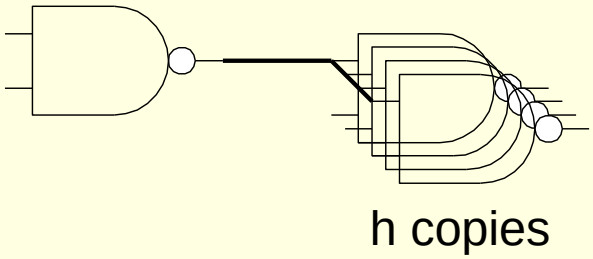




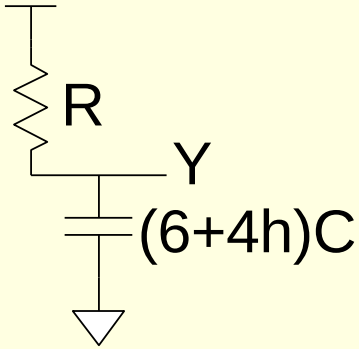
Propagation Delay

- Estimate worst-case rising and falling propagation delays of a 2-input NAND driving *h* identical gates.





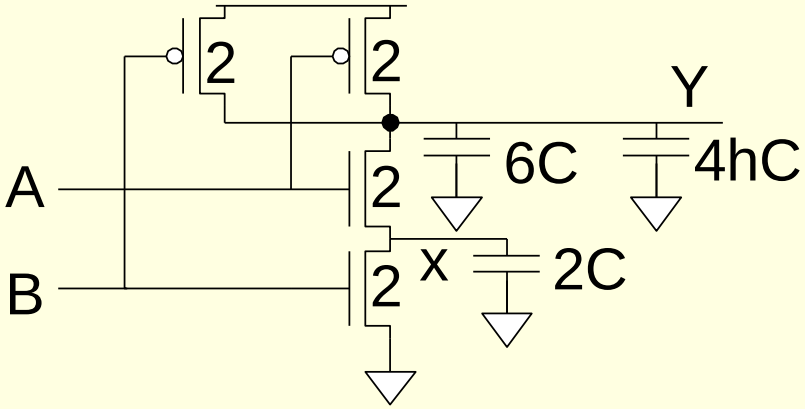
For the input combination : A=0&B=1

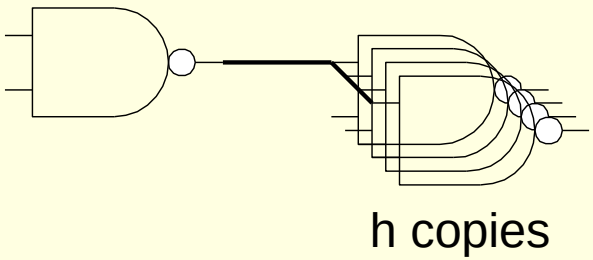


$$t_{pdr} =$$

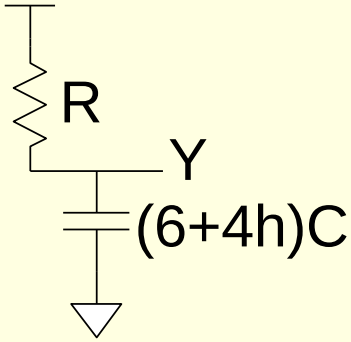
Propagation Delay

- Estimate worst-case rising and falling propagation delays of a 2-input NAND driving *h* identical gates.





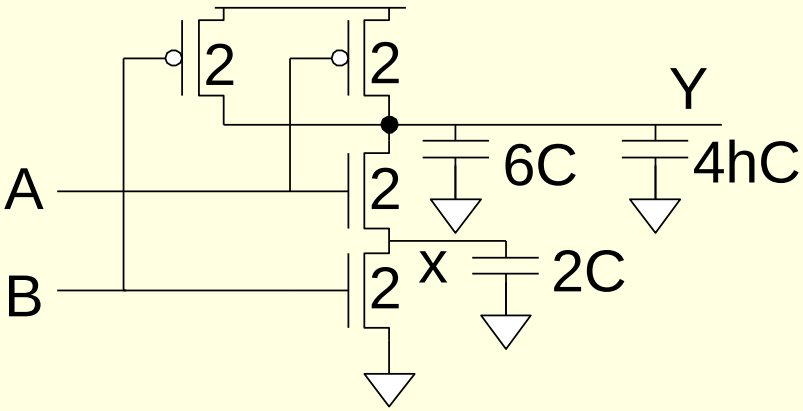
For the input combination : A=0&B=1

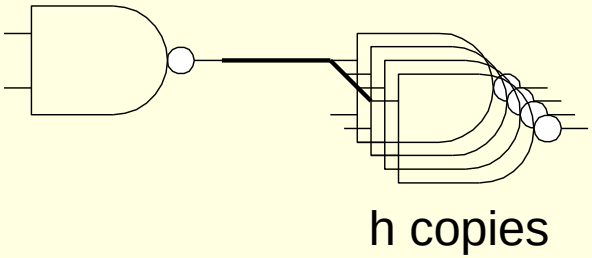


$$t_{pdr} = (6+4h)RC$$

Propagation Delay

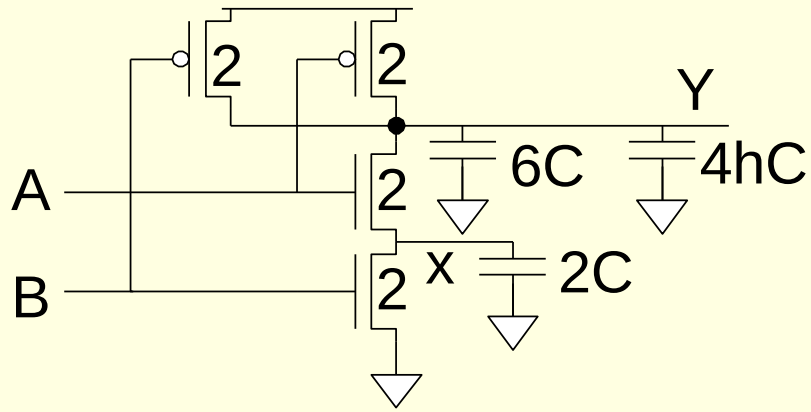
- Estimate worst-case rising and falling propagation delays of a 2-input NAND driving *h* identical gates.

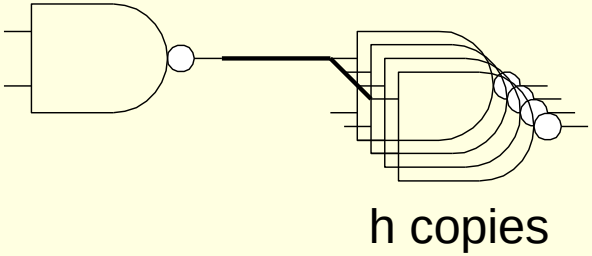


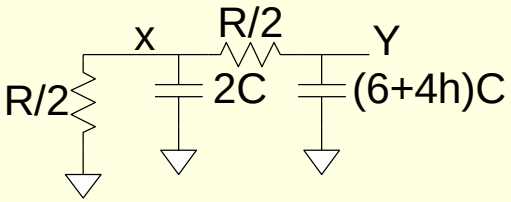


Propagation Delay

- Estimate worst-case rising and falling propagation delays of a 2-input NAND driving *h* identical gates.



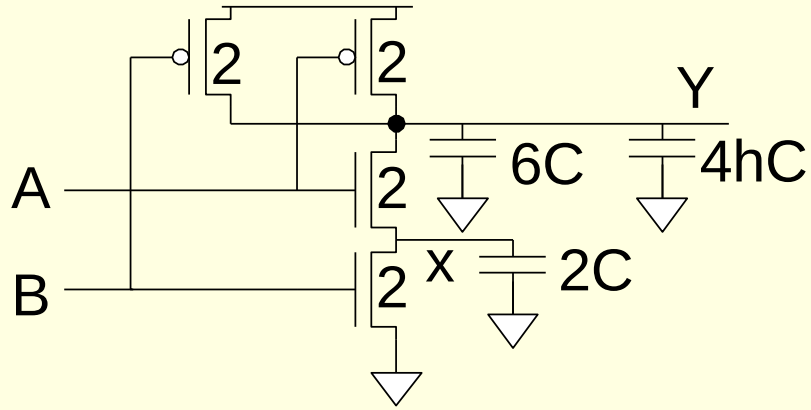


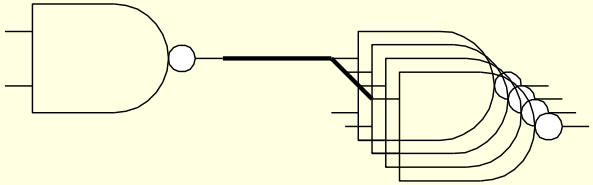


$$t_{pdf} =$$

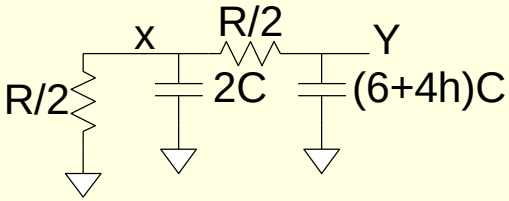
Propagation Delay

- Estimate worst-case rising and falling propagation delays of a 2-input NAND driving *h* identical gates.





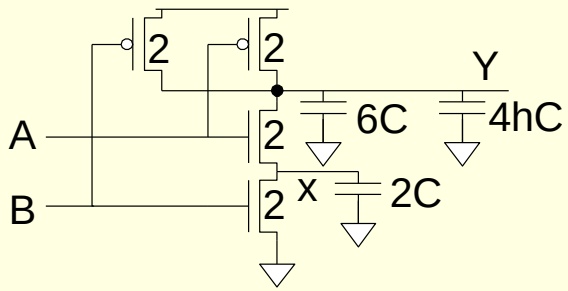
h copies

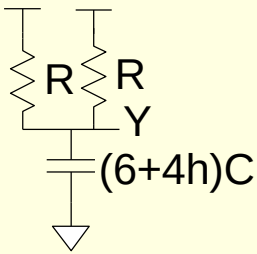


$$t_{pdf} = (2C)\left(\tfrac{R}{2}\right) + \left[(6+4h)C\right]\left(\tfrac{R}{2}+\tfrac{R}{2}\right)$$
$$= (7+4h)RC$$

Contamination Delay

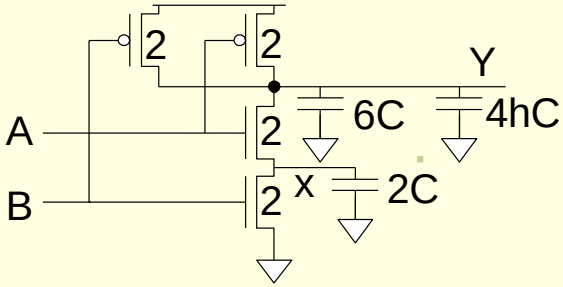
- Best-case (contamination) delay can be substantially less than propagation delay.
- Ex:(rising case) If both inputs fall simultaneously

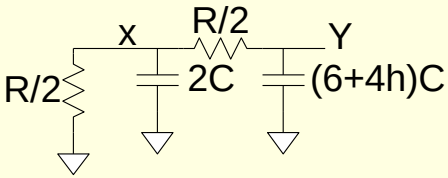




$$t_{cdr} = (3 + 2h)RC$$

- Ex:(falling case) If B = 1(already) and A ← 1(rising), node x is already discharged and thus could be ignored:





$$t_{cdf} = (6 + 4h)\,RC$$

Normalized Delay → Inverter

- Express delay in a process-independent form
- Circuits can be compared based on topology rather than speed of the manufacturing process
- Observe that the delay of an ideal fanout-of-1 inverter with no parasitic capacitance is $\tau = 3RC$

- Normalized Delay (d) relative to this inverter delay:

$$d = \frac{t_{pd}}{\tau}$$

- Hence, the delay of a fanout-of-h inverter can be written in normalized form as, (diffusion capacitance = gate capacitance)

$$d = h + 1$$

Linear Delay Model (LDM)

- RC model showed that delay is a linear function of fanout of a gate
- Designers simplify delay analysis by characterizing a gate by the slope and y-intercept of the function
- In LDM, normalized delay is expressed in terms of two components of delay:

$$d = f + p$$

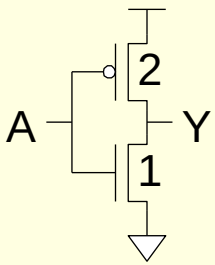
1. The parasitic delay (p) is the time for a gate to drive its own internal diffusion capacitance (no load)
2. The effort delay/stage effort (f) depends on h and g:

$$f = gh$$

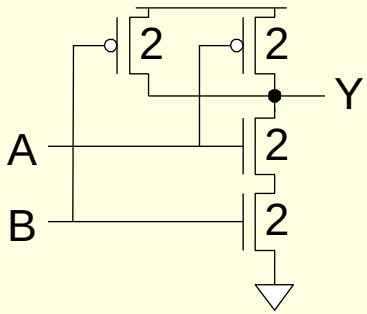
- h (fanout/electrical effort) is the ratio of the capacitance of the external load to input capacitance of the gate
- Complexity (logical effort, g) of the gate. Complex gates have greater logical efforts, indicating that they take longer to drive a given function

Computing Logical Effort

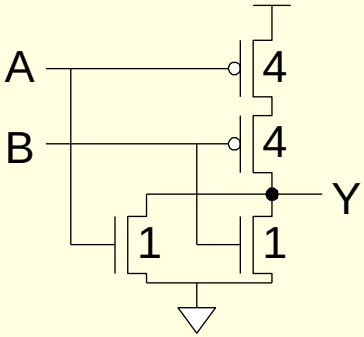
- DEF: *Logical effort is the ratio of the input capacitance of a gate to the input capacitance of an inverter delivering the same output current*.
- Measure from delay vs. fanout plots
- Or estimate by counting transistor widths



$C_{in} = 3$
$g = 3/3$



$C_{in} = 4$
$g = 4/3$



$C_{in} = 5$
$g = 5/3$

Catalog of Gates

- Logical effort of common gates

Gate type	Number of inputs				
	1	2	3	4	n
Inverter	1				
NAND		4/3	5/3	6/3	(n+2)/3
NOR		5/3	7/3	9/3	(2n+1)/3
Tristate / mux	2	2	2	2	2
XOR, XNOR		4, 4	6, 12, 6	8, 16, 16, 8	

Parasitic Delay

- The parasitic delay of a gate is the delay of the gate when it drives zero load. It can be estimated with RC delay models.
- The inverter has three units of diffusion capacitance on the output, so the parasitic delay is $3RC = \tau$
- The 3-input NAND and NOR each have 9 units of diffusion capacitance on the output, so the parasitic delay is three times as great.
- A crude method for hand calculations is to count only diffusion capacitance on the output node.

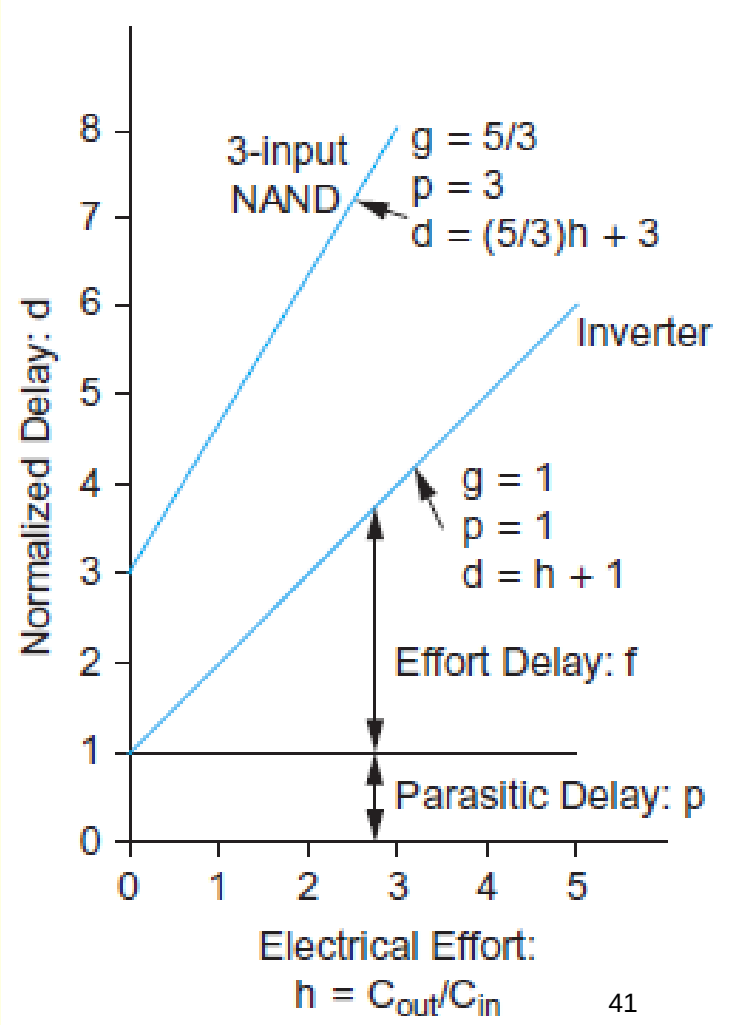
Catalog of Gates

- Parasitic delay of common gates
  - In multiples of p_{inv} (≈ 1)

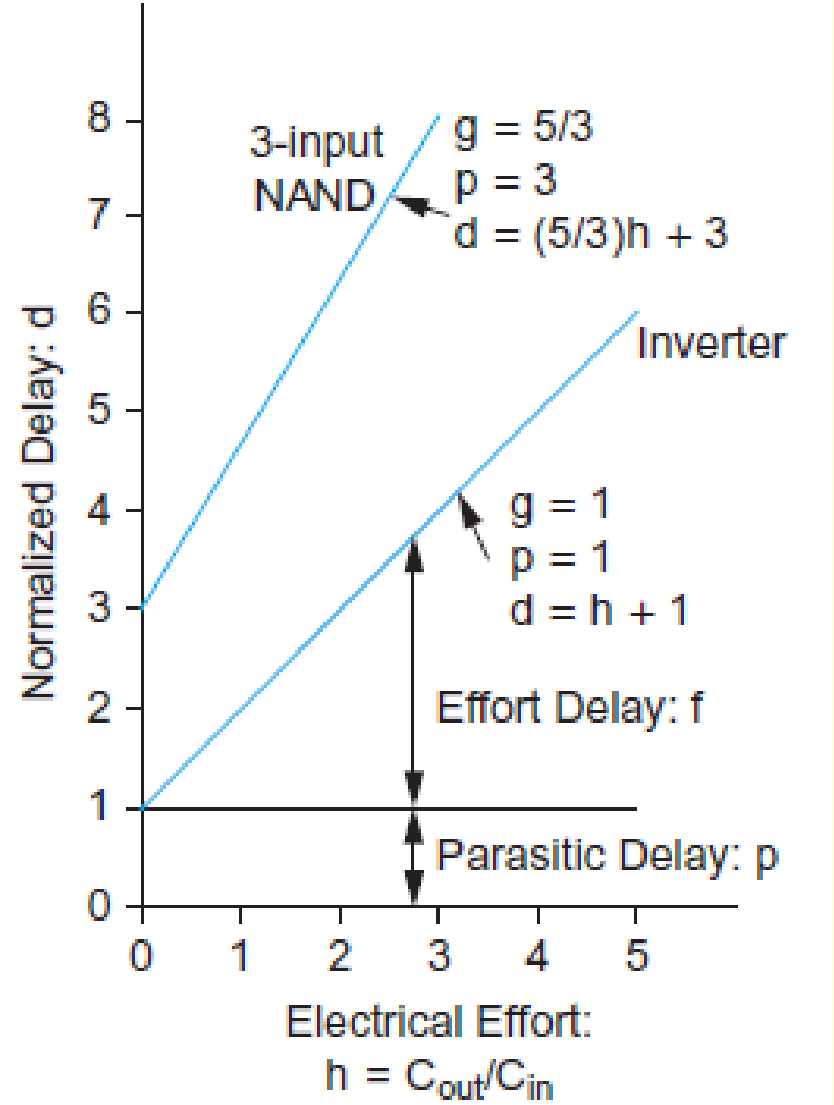
Gate type	Number of inputs				
	1	2	3	4	n
Inverter	1				
NAND		2	3	4	n
NOR		2	3	4	n
Tristate / mux	2	4	6	8	2n
XOR, XNOR		4	6	8	

Linear Delay Model...continue

- Figure plots the normalized delay vs. electrical effort for an idealized inverter and 3-input NAND gate.
- The y-intercepts indicate the parasitic delay, i.e., the delay when the gate drives no load.
- The slope of the lines is the logical effort. The inverter has a slope of 1 by definition. The NAND has a slope of 5/3.

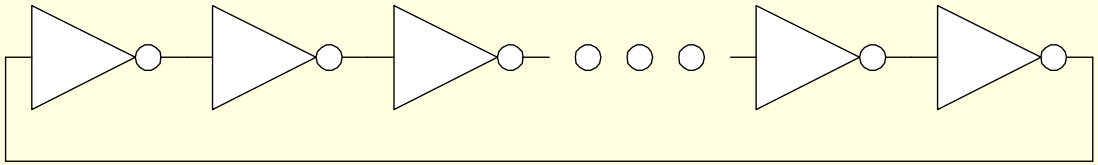


Linear Delay Model...continue



Example: Ring Oscillator…odd no. of inverters

- Estimate the frequency of an N-stage ring oscillator



Logical Effort: g =

Electrical Effort: h =

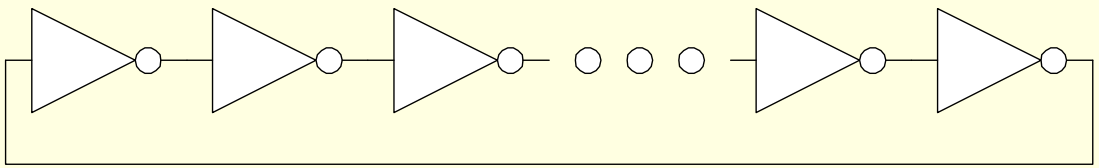
Parasitic Delay: p =

Stage Delay: d =

Frequency: f_{osc} =

Example: Ring Oscillator…odd no. of inverters

- Estimate the frequency of an N-stage ring oscillator



Logical Effort: $g = 1$

Electrical Effort: $h = 1$

Parasitic Delay: $p = 1$

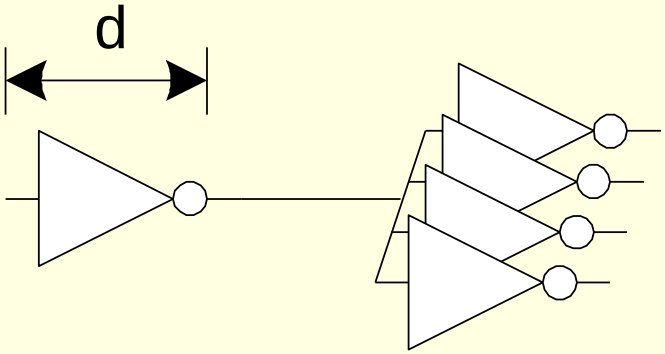
Stage Delay: $d = 2$

Frequency: $f_{osc} = 1/(2*N*d) = 1/4N$

31 stage ring oscillator in 0.6 μm process has frequency of ~ 200 MHz

Example: FO4 Inverter

- Estimate the delay of a fanout-of-4 (FO4) inverter



Logical Effort: g =

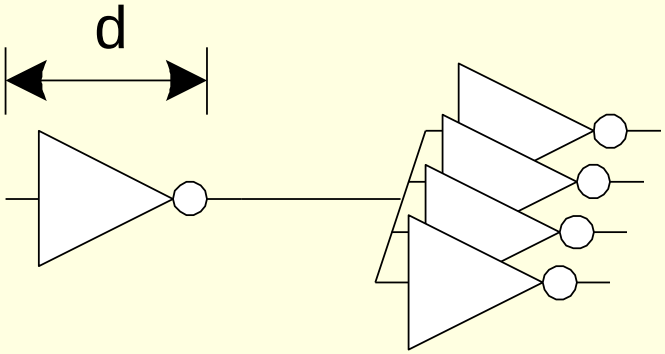
Electrical Effort: h =

Parasitic Delay: p =

Stage Delay: d =

Example: FO4 Inverter

- Estimate the delay of a fanout-of-4 (FO4) inverter



Logical Effort: g = 1

Electrical Effort: h = 4

Parasitic Delay: p = 1

Stage Delay: d = 5

The FO4 delay is about

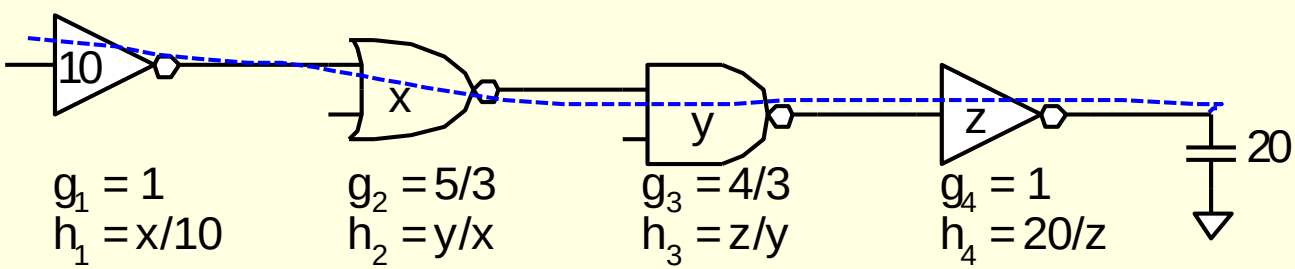
200 ps in 0.6 μm process

60 ps in a 180 nm process

f/3 ns in an *f* μm process

Multistage Logic Networks

- Logical effort generalizes to multistage networks
- *Path Logical Effort* $G = \prod g_i$
- *Path Electrical Effort* $H = \dfrac{C_{\text{out-path}}}{C_{\text{in-path}}}$
- *Path Effort* $F = \prod f_i = \prod g_i h_i$

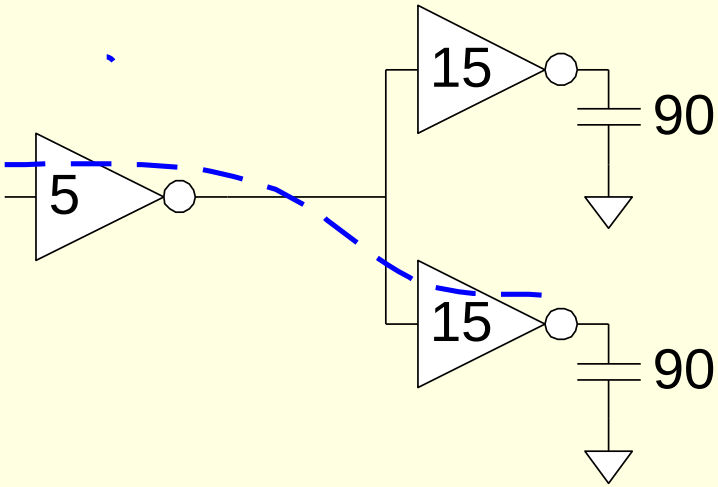


Multistage Logic Networks

- Logical effort generalizes to multistage networks
- *Path Logical Effort* $G = \prod g_i$
- *Path Electrical Effort* $H = \dfrac{C_{\text{out-path}}}{C_{\text{in-path}}}$
- *Path Effort* $F = \prod f_i = \prod g_i h_i$
- Can we write F = GH?

Paths that Branch

- No! Consider paths that branch:



G =

H =

GH =

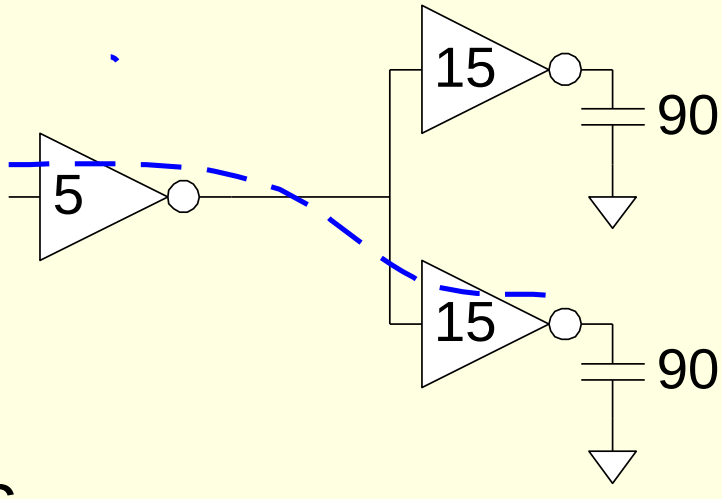
h_1 =

h_2 =

F = GH?

Paths that Branch

- No! Consider paths that branch:



$G = 1$

$H = 90 / 5 = 18$

$GH = 18$

$h_1 = (15 + 15) / 5 = 6$

$h_2 = 90 / 15 = 6$

$F = g_1 g_2 h_1 h_2 = 36 = 2GH$

Branching Effort

- Introduce *branching effort*
  - Accounts for branching between stages in path

$$b = \frac{C_{\text{on path}} + C_{\text{off path}}}{C_{\text{on path}}}$$

  - Path branching effort *B*

$$B = \prod b_i$$

- Now we compute the path effort
  - F = GBH

Multistage Delays

- Path Effort Delay $$D_F = \sum f_i$$
- Path Parasitic Delay $$P = \sum p_i$$
- Path Delay $$D = \sum d_i = D_F + P$$

Designing Fast Circuits

$$D = \sum d_i = D_F + P$$

- Delay is smallest when each stage bears same effort

$$\hat{f} = g_i h_i = F^{\frac{1}{N}}$$

- Thus minimum delay of N stage path is

$$\boxed{D = NF^{\frac{1}{N}} + P}$$

- This is a key result of logical effort
  - Find fastest possible delay
  - Doesn't require calculating gate sizes

Gate Sizes

- How wide should the gates be for least delay?

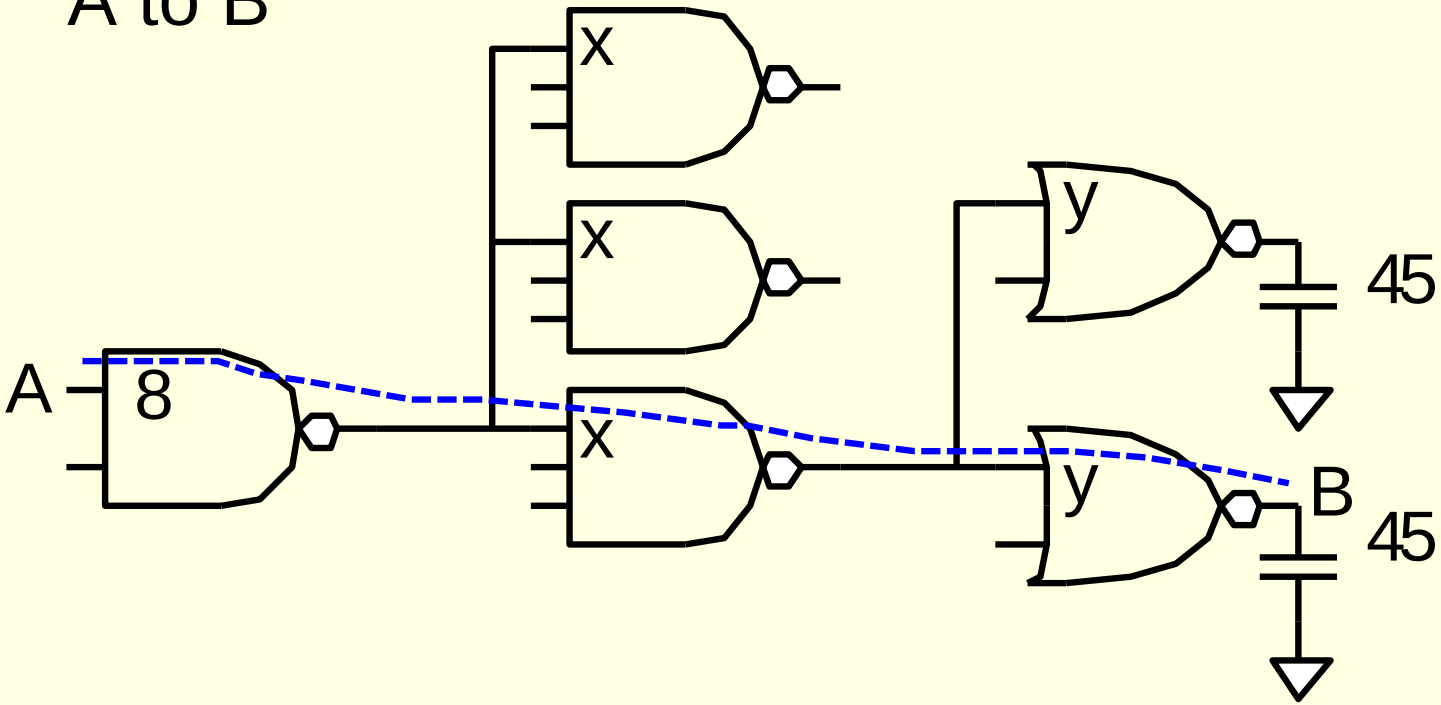
$$\hat{f} = gh = g\frac{C_{out}}{C_{in}}$$

$$\Rightarrow C_{in_i} = \frac{g_i C_{out_i}}{\hat{f}}$$

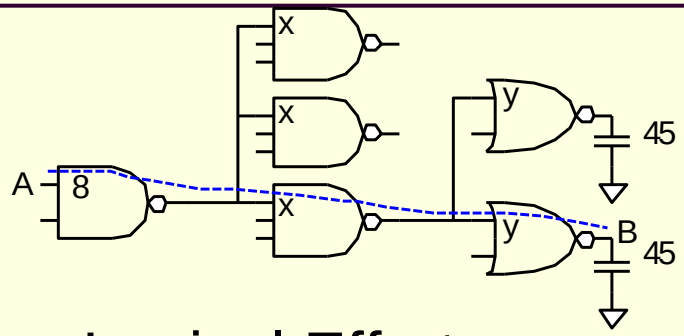
- Working backward, apply capacitance transformation to find input capacitance of each gate given load it drives.
- Check work by verifying input cap spec is met.

Example: 3-stage path

- Select gate sizes x and y for least delay from A to B



Example: 3-stage path



Logical Effort G =

Electrical Effort H =

Branching Effort B =

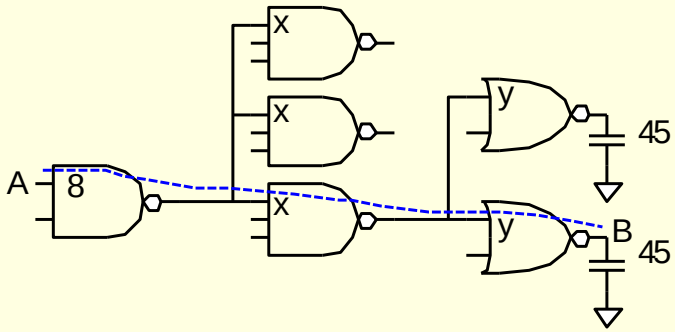
Path Effort F =

Best Stage Effort $\hat{f} =$

Parasitic Delay P =

Delay D =

Example: 3-stage path



Logical Effort	G = (4/3)*(5/3)*(5/3) = 100/27
Electrical Effort	H = 45/8
Branching Effort	B = 3 * 2 = 6
Path Effort	F = GBH = 125
Best Stage Effort	$\hat{f} = \sqrt[3]{F} = 5$
Parasitic Delay	P = 2 + 3 + 2 = 7
Delay	D = 3*5 + 7 = 22 = 4.4 FO4

Example: 3-stage path

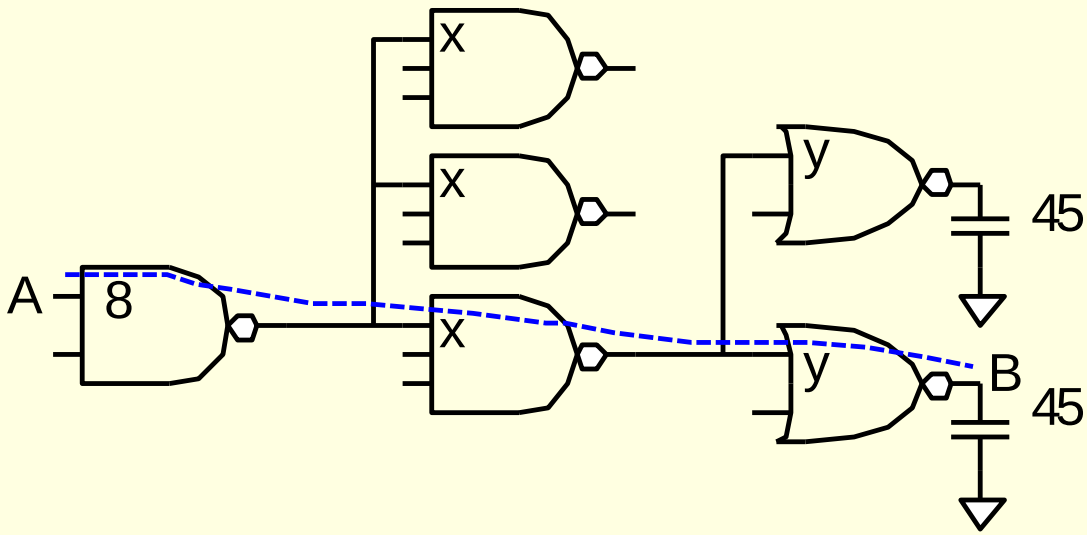
- Work backward for sizes

  y =

  x =

$$\hat{f} = gh = g\frac{C_{out}}{C_{in}}$$

$$\Rightarrow C_{in_i} = \frac{g_i C_{out_i}}{\hat{f}}$$



Example: 3-stage path

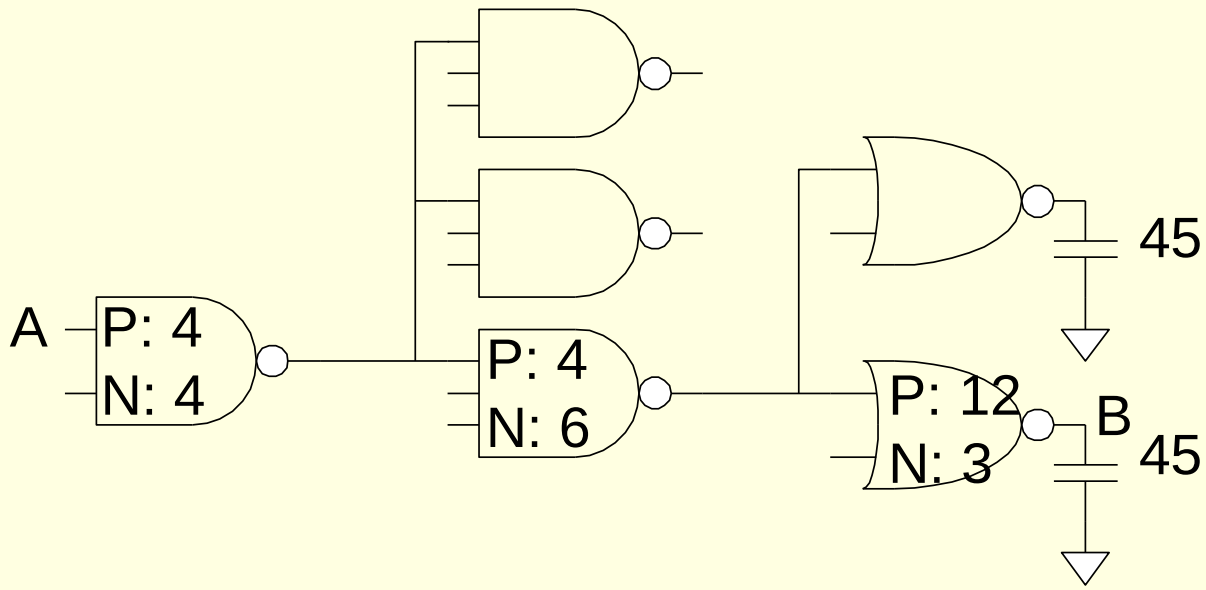
- Work backward for sizes

  y = 45 * (5/3) / 5 = 15

  x = (15*2) * (5/3) / 5 = 10

$$\hat{f} = gh = g\frac{C_{out}}{C_{in}}$$

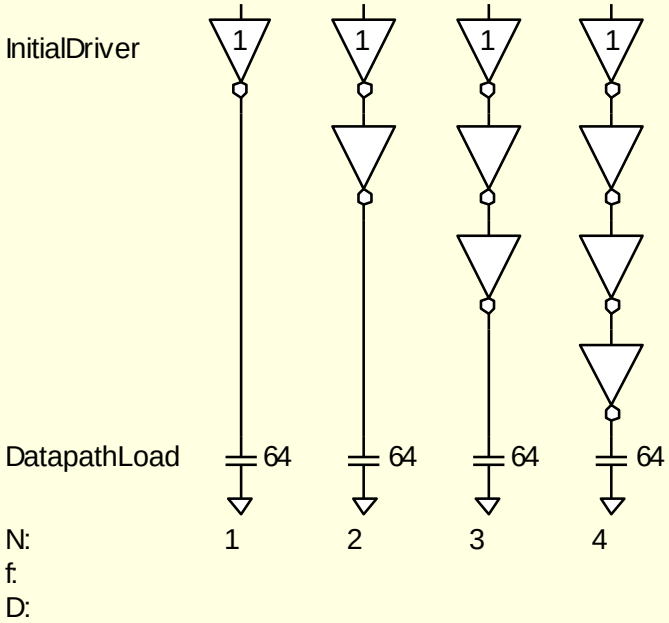
$$\Rightarrow C_{in_i} = \frac{g_i C_{out_i}}{\hat{f}}$$



Best Number of Stages

- How many stages should a path use?
  - Minimizing number of stages is not always fastest
- Example: drive 64-bit datapath with unit inverter

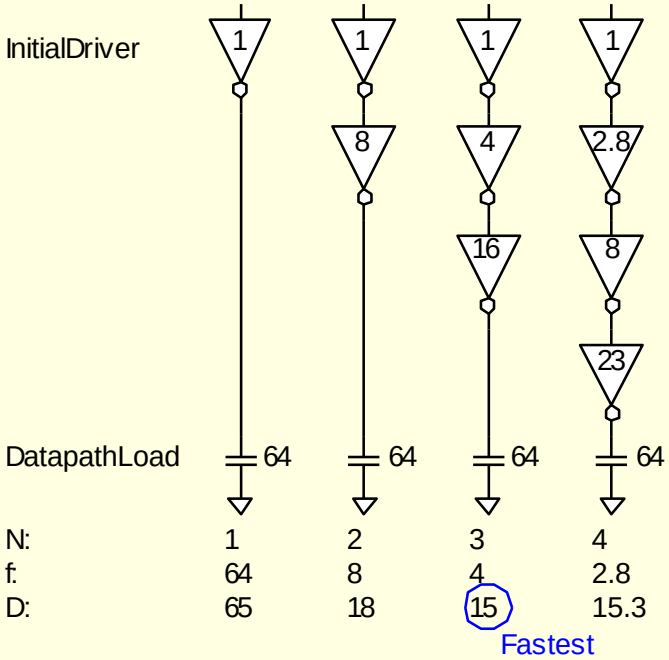
D =



Best Number of Stages

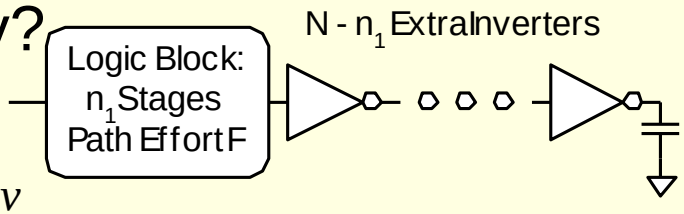
- How many stages should a path use?
  - Minimizing number of stages is not always fastest
- Example: drive 64-bit datapath with unit inverter

$$D = NF^{1/N} + P$$
$$= N(64)^{1/N} + N$$



Derivation

- Consider adding inverters to end of path
  - How many give least delay?



$$D = NF^{\frac{1}{N}} + \sum_{i=1}^{n_1} p_i + \left(N - n_1\right) p_{inv}$$

$$\frac{\partial D}{\partial N} = -F^{\frac{1}{N}} \ln F^{\frac{1}{N}} + F^{\frac{1}{N}} + p_{inv} = 0$$

- Define best stage effort $\rho = F^{\frac{1}{N}}$

$$p_{inv} + \rho\left(1 - \ln \rho\right) = 0$$

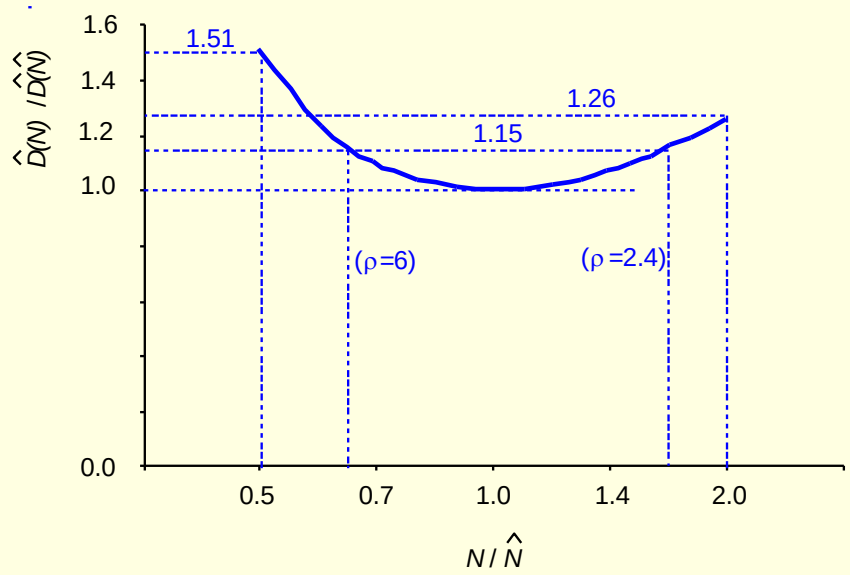
Best Stage Effort

- $p_{inv} + \rho\left(1 - \ln \rho\right) = 0$ has no closed-form solution

- Neglecting parasitics (p_{inv} = 0), we find ρ = 2.718 (e)
- For p_{inv} = 1, solve numerically for ρ = 3.59

Sensitivity Analysis

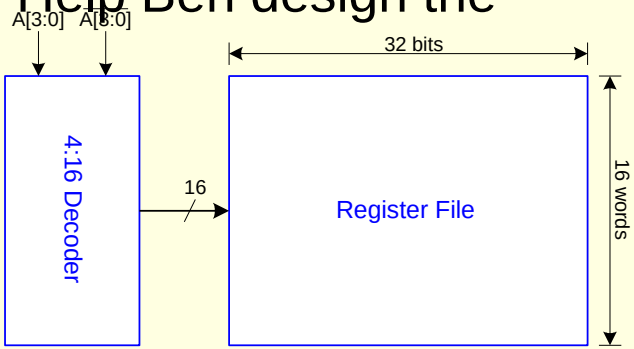
- How sensitive is delay to using exactly the best number of stages?



- $2.4 < \rho < 6$ gives delay within 15% of optimal
  - We can be sloppy!
  - I like $\rho = 4$

Example, Revisited

- Ben Bitdiddle is the memory designer for the Motoroil 68W86, an embedded automotive processor. Help Ben design the decoder for a register file.



- Decoder specifications:
  - 16 word register file
  - Each word is 32 bits wide
  - Each bit presents load of 3 unit-sized transistors
  - True and complementary address inputs A[3:0]
  - Each input may drive 10 unit-sized transistors
- Ben needs to decide:
  - How many stages to use?
  - How large should each gate be?
  - How fast can decoder operate?

Number of Stages

- Decoder effort is mainly electrical and branching

  Electrical Effort: H =

  Branching Effort: B =

- If we neglect logical effort (assume G = 1)

  Path Effort: F =

  Number of Stages: N =

Number of Stages

- Decoder effort is mainly electrical and branching

  Electrical Effort: $H = (32*3) / 10 = 9.6$

  Branching Effort: $B = 8$

- If we neglect logical effort (assume $G = 1$)

  Path Effort: $F = GBH = 76.8$

  Number of Stages: $N = \log_4 F = 3.1$

- Try a 3-stage design

Gate Sizes & Delay

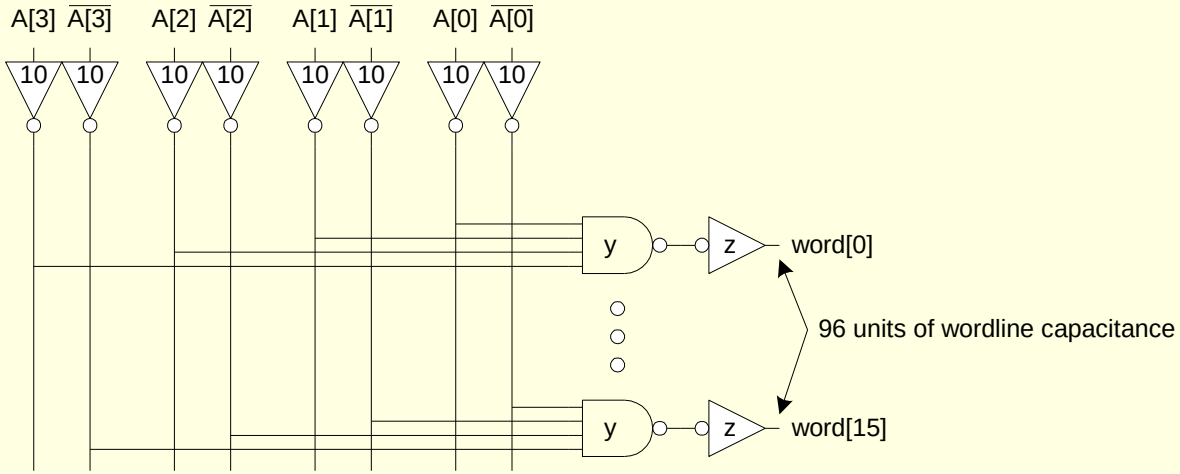
Logical Effort: G =

Path Effort: F =

Stage Effort: $\hat{f} =$

Path Delay: $D =$

Gate sizes: z = y =



Gate Sizes & Delay

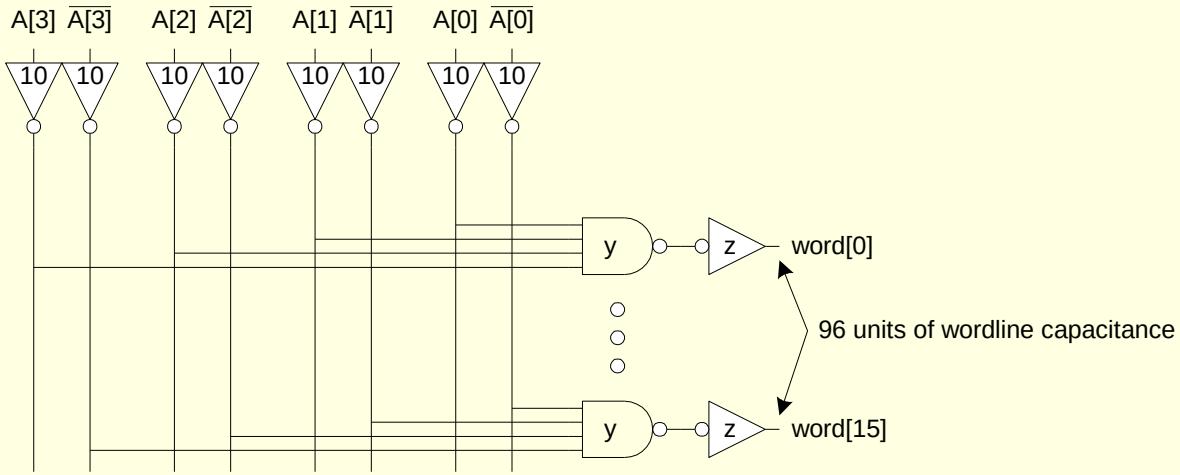
Logical Effort: G = 1 * 6/3 * 1 = 2

Path Effort: F = GBH = 154

Stage Effort: $\hat{f} = F^{1/3} = 5.36$

Path Delay: $D = 3\hat{f} + 1 + 4 + 1 = 22.1$

Gate sizes: z = 96*1/5.36 = 18 y = 18*2/5.36 = 6.7



Comparison

- Compare many alternatives with a spreadsheet

Design	N	G	P	D
NAND4-INV	2	2	5	29.8
NAND2-NOR2	2	20/9	4	30.1
INV-NAND4-INV	3	2	6	22.1
NAND4-INV-INV-INV	4	2	7	21.1
NAND2-NOR2-INV-INV	4	20/9	6	20.5
NAND2-INV-NAND2-INV	4	16/9	6	19.7
INV-NAND2-INV-NAND2-INV	5	16/9	7	20.4
NAND2-INV-NAND2-INV-INV-INV	6	16/9	8	21.6

Review of Definitions

Term	Stage	Path
number of stages	1	N
logical effort	g	$G = \prod g_i$
electrical effort	$h = \frac{C_{\text{out}}}{C_{\text{in}}}$	$H = \frac{C_{\text{out-path}}}{C_{\text{in-path}}}$
branching effort	$b = \frac{C_{\text{on-path}} + C_{\text{off-path}}}{C_{\text{on-path}}}$	$B = \prod b_i$
effort	$f = gh$	$F = GBH$
effort delay	f	$D_F = \sum f_i$
parasitic delay	p	$P = \sum p_i$
delay	$d = f + p$	$D = \sum d_i = D_F + P$

Method of Logical Effort

1) Compute path effort
2) Estimate best number of stages
3) Sketch path with N stages
4) Estimate least delay
5) Determine best stage effort
6) Find gate sizes

$$F = GBH$$

$$N = \log_4 F$$

$$D = NF^{\frac{1}{N}} + P$$

$$\hat{f} = F^{\frac{1}{N}}$$

$$C_{in_i} = \frac{g_i C_{out_i}}{\hat{f}}$$

Limits of Logical Effort

- Chicken and egg problem
  - Need path to compute G
  - But don’t know number of stages without G
- Simplistic delay model
  - Neglects input rise time effects
- Interconnect
  - Iteration required in designs with wire
- Maximum speed only
  - Not minimum area/power for constrained delay

Summary

- Logical effort is useful for thinking of delay in circuits
  - Numeric logical effort characterizes gates
  - NANDs are faster than NORs in CMOS
  - Paths are fastest when effort delays are ~4
  - Path delay is weakly sensitive to stages, sizes
  - But using fewer stages doesn't mean faster paths
  - Delay of path is about $\log_4 F$ FO4 inverter delays
  - Inverters and NAND2 best for driving large caps
- Provides language for discussing fast circuits
  - But requires practice to master